

## Data curation

When it comes to procuring data for fine-tuning, the approaches differ between task-specific and domain-specific models. For task-specific models, data curation often involves collecting examples of the desired task from existing datasets or creating new ones. This might involve gathering pairs of original and summarized texts for a summarization model or collecting sentences in different languages for a translation model.

Domain-specific data curation can be more challenging. It often requires collaboration with subject matter experts to gather and validate relevant texts, research papers, technical documents, and other domain-specific content. In some cases, it may involve partnering with organizations or institutions that have access to large repositories of specialized information. The quality and relevance of this data is crucial, as it directly impacts the model's ability to understand and generate content in the target domain.

It's worth noting that few-shot prompting has emerged as an alternative strategy to fine-tuning, especially for task-specific applications. This approach leverages the capabilities of large, powerful models by providing a few examples of the desired task within the input prompt. While not a replacement for fine-tuning in all scenarios (e.g., when you want to learn a new domain), few-shot prompting can be an efficient way to adapt models to new tasks without the need for extensive additional training.

In practice, the line between task-specific and domain-specific models can sometimes blur. For instance, a model fine-tuned for medical diagnosis could be considered both task-specific (focused on diagnosis) and domain-specific (specialized in medical knowledge). The key is to understand the primary goal of the fine-tuning process and tailor the approach accordingly.

At this point in the process, we should have a collection of datasets suited for our use case. The next step consists of refining the quality of the samples through rule-based filtering, data duplication, data decontamination, and data quality evaluation.

## Rule-based filtering

Rule-based filtering is a systematic approach to data quality control that relies on explicit, pre-defined rules to evaluate and filter data samples. These rules are typically designed to address common quality issues and can range from simple checks to more complex logical operations. The primary goal of rule-based filtering is to maintain a high standard of data quality by removing samples that do not meet specific criteria.

**Length filtering** is a straightforward yet effective rule-based filtering technique. This method involves setting thresholds for the acceptable length of responses in the dataset. Extremely short responses often lack sufficient information to be meaningful, while excessively long ones may contain irrelevant or redundant content. It's important to note that the appropriate length thresholds can vary significantly depending on the specific task and domain. For example, a dataset for generating concise summaries might have a lower maximum threshold compared to one for detailed explanations.

**Keyword exclusion** is another powerful rule-based filtering technique that focuses on the content of the samples rather than their structure. This method involves creating a list of keywords or phrases associated with low-quality or inappropriate content, and then filtering out any samples that contain these terms. The keyword list can include obvious indicators of low quality, such as profanities or spam-related terms, as well as domain-specific words that might indicate irrelevant or off-topic content. For instance, in a dataset for a professional writing assistant, you might exclude samples containing slang terms or informal expressions that don't align with the intended tone and style.

**Format checking** is recommended for datasets that include structured data or follow specific formatting requirements. This technique ensures that all samples adhere to the expected format, maintaining consistency and facilitating processing downstream. Format checking can be particularly important for datasets containing code samples, JSON structures, or other formatted text. For example, in a dataset of programming instructions and solutions, you might implement rules to verify that code samples are syntactically correct and follow specified style guidelines.

Rule-based filtering offers significant advantages in preparing instruction datasets. Its speed and efficiency allow for rapid application to large volumes of data, making it highly scalable. The consistency of rule application ensures uniform treatment of data, reducing human error and bias. Furthermore, the explicit definition of filtering criteria provides transparency and interpretability, facilitating easy understanding, auditing, and adjustment. The ability to automate rule-based filtering reduces the need for manual intervention and enables continuous data quality monitoring.

However, rule-based filtering also has limitations that must be considered. Predefined rules may lack the nuance required to capture the full complexity of language and context, potentially leading to the removal of valid but unusual samples. The typically binary nature of rules (pass/fail) may not always align with the nuanced nature of language and instruction quality. Additionally, as data patterns and quality standards evolve, rules need regular review and updates to remain effective. There's also a risk that poorly designed rules could inadvertently introduce or amplify biases in the dataset.

## Data deduplication

Dataset diversity is fundamental to training models that can generalize well to new, unseen data. When a dataset contains duplicates or near-duplicates, it can lead to several issues:

- **Overfitting:** Models may memorize specific examples rather than learning general patterns.
- **Biased performance:** Overrepresented data points may skew the model's performance towards certain types of inputs.
- **Inefficient training:** Redundant data can increase training time without providing additional valuable information.
- **Inflated evaluation metrics:** Duplicate data in test sets may lead to overly optimistic performance estimates.

To deduplicate datasets, we distinguish between exact and fuzzy deduplication. **Exact deduplication** removes identical samples through a straightforward process involving data normalization, hash generation, and duplicate removal. Data normalization standardizes the format of entries, such as converting text to lowercase. Hash generation then creates unique hashes for each entry using algorithms like MD5 or SHA-256. These hashes are compared to find matches, and duplicates are removed, leaving only one instance of each. While effective for identical entries, exact deduplication does not detect near-duplicates or semantically similar content, requiring more advanced techniques for those cases.

The most popular approach to **fuzzy deduplication** is MinHash deduplication. Compared to other fuzzy techniques, it maintains high accuracy while significantly reducing computational complexity. MinHash operates by generating compact representations, or signatures, for each data item. These signatures serve as fingerprints that capture the essence of the data while drastically reducing its dimensionality. In practice, MinHash transforms data items (such as text documents) into sets of shingles, applies multiple hash functions to these sets, and selects the minimum hash values to form signature vectors. These signatures can then be compared using similarity measures like Jaccard similarity to efficiently identify near-duplicates.

In addition to exact and fuzzy deduplication, **semantic similarity** takes a different approach by focusing on the meaning of text for deduplication. This method involves converting words or entire samples into vector representations using various natural language processing techniques. Word embedding models such as Word2Vec, GloVe, and FastText transform individual words into dense vectors, capturing semantic relationships.

For more context-aware representations, language models like BERT, sentence transformers, or cross-encoders can generate embeddings for entire sentences or documents. Once these vector representations are obtained, deduplication can be performed by comparing the similarity between vectors. Common similarity measures include cosine similarity or Euclidean distance. Samples with high similarity scores above a predefined threshold can be considered duplicates. For large datasets, clustering techniques may be applied to group similar vectors. Methods like K-means, DBSCAN, or hierarchical clustering can efficiently organize the vector space, allowing for the identification of clusters that represent semantically similar content. Within each cluster, a representative sample can be retained while others are marked as duplicates.

## Data decontamination

Data decontamination is the process of ensuring that the training dataset does not contain samples that are identical or highly similar to those in the evaluation or test sets. This step is important for ensuring the quality of the model evaluation and preventing overfitting or memorization of test data.

Data decontamination uses techniques from data deduplication. Exact matching can be used to remove any training samples that are identical to those in the evaluation sets. This can be done using hash functions or direct string comparisons. Next, we can also use near-duplicate detection methods to identify and remove training samples that are very similar to evaluation samples, even if they are not exactly the same. This often involves techniques like MinHash or computing similarity scores based on n-grams or embeddings.



A simple way to perform data decontamination is to add your evaluation set to the instruction dataset during the data deduplication stage. In this case, we want to ensure that we only remove samples from the instruction dataset, which can be implemented in different ways (only filtering out the first duplicate, recording the indexes of the evaluation samples, etc.). Ideally, you can automatically add your evaluation sets in the data deduplication stage to fully automate this process. This is particularly efficient if you iterate over several versions of custom benchmarks.

Another aspect of data decontamination is filtering out samples that may have been derived from the same source as evaluation data. This can involve checking for overlapping phrases, similar sentence structures, or common metadata. Practitioners may also use provenance tracking (source the data they use) to identify and exclude data from specific sources that are known to be used in evaluation sets.

## Data quality evaluation

Data quality evaluation is a critical aspect of machine learning, particularly for LLMs. The process involves assessing various characteristics of datasets, including accuracy, diversity, and complexity. While some aspects like mathematical accuracy can be easily verified using tools such as Python interpreters, evaluating subjective or open-ended content remains challenging.

Traditional methods of data quality assessment include human annotation, which generally provides high accuracy but is resource-intensive. To address scalability issues, machine learning techniques have been developed to automate the evaluation process. These include using LLMs as judges, reward models, and classifiers trained for quality prediction.

**The LLM-as-a-judge** strategy involves prompting LLMs to evaluate the quality of each sample. This approach has become popular due to its flexibility and ease of use, though it does present some challenges. Different LLMs have different levels of performance across tasks, and their evaluations often align more closely with those of non-experts. With domain-specific datasets, you might want to use domain-specific models instead of better, general-purpose LLMs. Comparative assessment methods (e.g., “Is answer A better than answer B?”) generally outperform absolute scoring approaches (e.g., “Rate answer A between 1 and 4”), though both can be used at scale with sufficient prompt engineering. We recommend iterating through different prompts over a representative subset to manually verify the quality of the responses. *Table 5.2* shows an example of a custom prompt for a judge LLM.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Instruction</b></p> <p>You are a data quality evaluator. Your goal is to assess an instruction and its corresponding answer, determining how effectively the answer addresses the given task.</p> <p>In your evaluation, you will provide feedback detailing the strengths and weaknesses of the answer, followed by a score on a scale of 1 to 4.</p> <p>A score of 1 means that the answer is terrible and irrelevant to the instruction.</p> <p>A score of 2 means that the answer is not helpful and misses important aspects of the instruction.</p> <p>A score of 3 means that the answer is helpful but could be improved in terms of relevance, accuracy, and depth.</p> <p>A score of 4 means that the answer is excellent and fully addresses the task.</p> <p>Provide your evaluation as follows:</p> <p>Feedback: (strengths and weaknesses you find relevant)</p> <p>Score: (number between 1 and 4)</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Table 5.2 – Example of LLM-as-a-judge prompt for data quality evaluation

LLM-as-a-judge is known to have several biases. First, it has a position bias in comparative scoring, where the LLM judge favors the first answer. This can be addressed by randomizing the order of answers A and B. In addition, like humans, LLM judges favor long answers. Length normalization techniques can be applied to absolute scoring to mitigate this issue. Finally, LLM judges are known to have intra-model favoritism, meaning that they prefer models from the same family (GPT-4o with GPT-4 and GPT-4o mini, for example). This can be addressed by using several models instead of a single one.

In general, to improve evaluation reliability, strategies such as using multiple LLMs as a jury reduce bias and improve consistency. Leveraging a jury of smaller LLMs can also reduce costs while increasing accuracy and mitigating intra-model favoritism. For specific applications like chatbots, it's advisable to aim for high agreement between LLM judges and human evaluators (around 80%). Simple grading scales (with few-shot prompting) and task-specific benchmarks are also recommended to ensure relevant and interpretable evaluations.

**Reward models** are another way to re-purpose LLMs for data quality evaluation. The term “reward model” comes from Reinforcement Learning from Human Feedback (RLHF, see *Chapter 6*). They can be broadly defined as models that take an instruction and answer pair and return a score as output. Generally, reward models are created by adding a linear head on top of a decoder-only architecture like Gemma or Llama. They are then trained for this specific purpose, using either reinforcement learning or traditional fine-tuning. *Figure 5.3* shows ArmoRM-Llama3-8B-v0.1's architecture, which adds regression and gating layers on top of a Llama 3 8B model. This model outputs multiple scores to target specific dimensions, such as helpfulness, correctness, coherence, complexity, and verbosity. This allows for a more fine-grained approach to data quality evaluation.

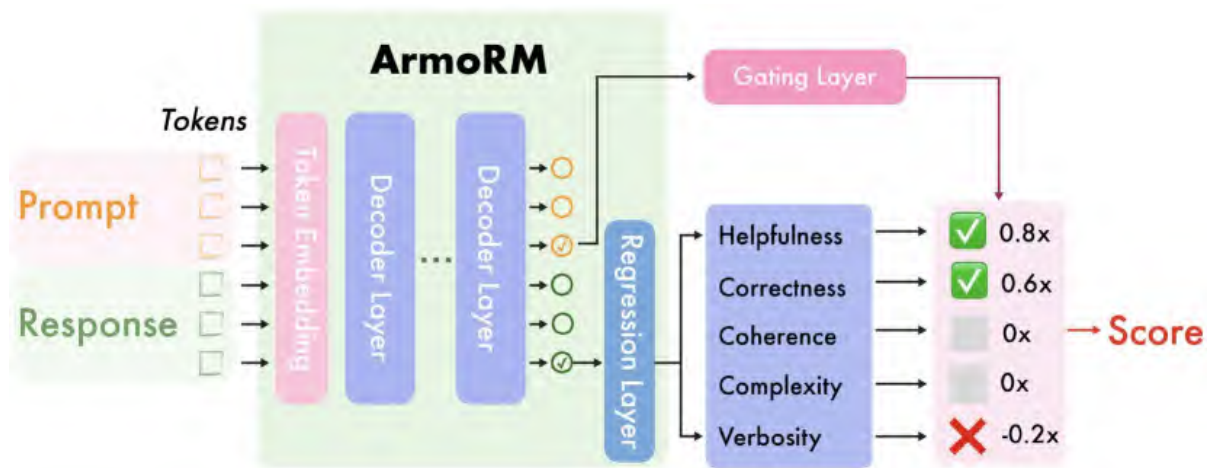


Figure 5.3 – Architecture of RLHFlow/ArmoRM-Llama3-8B-v0.1, based on Llama 3 (Source: <https://doi.org/10.48550/arXiv.2406.12845>)